AUTOMIND

CONFIGURACIÓN MANUAL / GOOGLE CLOUD

React Native + TypeScript | PostgreSQL y cuatro tablas | 3 de noviembre de 2026

1. Qué incluye esta versión

Una sola base PostgreSQL llamada storeml_database contiene exactamente cuatro tablas de la aplicación: usuarios, modelos, permisos y solicitudes. La API está escrita únicamente en Python, con FastAPI, SQLAlchemy, pg8000 y el conector oficial de Cloud SQL. Utiliza consultas SQL parametrizadas. TypeScript se utiliza en la app móvil. La app móvil conserva React Native, TypeScript, Skia, RaTeX y login únicamente Google Gmail.

Los 24 modelos siguen siendo calculadoras: Input 1 + Input 2 + Input 3. Admiten negativos y decimales, con rango de -1.000.000.000.000 a 1.000.000.000.000 por input. Sus IDs son enteros únicos de ocho dígitos, del 10000001 al 10000024. Los nombres KS, CART, Red Neuronal, Regresión, Holt Winters y Prophet son etiquetas; todos suman.

Los archivos están organizados primero por modelos y luego por área: inference/modelos/inmobiliario/cart.py, por ejemplo. Cada Python declara MODEL_ID y contiene su propia función calcular. En la tabla modelos, la columna archivo guarda la ruta relativa desde la raíz del repositorio público de GitHub.

GitHub aporta las fuentes durante el build. Cloud Run ejecuta la copia del código incluida en su contenedor. El ID se resuelve mediante un registro fijo de módulos permitidos; cambiar la columna archivo por sí sola no ejecuta otro archivo. La app no envía rutas ni credenciales de GitHub.

El archivo sql/02-catalogo-y-accesos.sql prepara 24 fichas y 12 permisos para artemioaday@gmail.com: KS, CART y Red Neuronal por área. Tu registro de usuario se crea al primer login Google real. Los permisos previos quedan reservados por Gmail y se vinculan al subject verificado al registrarte.

El despliegue automático se ejecuta con GitHub Actions al actualizar main, una vez configurados OIDC y las variables. Las pull requests ejecutan pruebas sin desplegar. Las imágenes se compilan en GitHub, se publican en Artifact Registry y se despliegan en Cloud Run. La configuración inicial se realiza con Google Cloud Console, Cloud SQL Studio y GitHub web. En Studio pegarás SQL y pulsarás Ejecutar; no necesitas Cloud Shell. La sección Android utiliza herramientas locales para compilar.

2. Proyecto y cuentas de servicio

En Google Cloud selecciona tu proyecto y vincula facturación. Anota su ID y su número (son distintos). En APIs y servicios > Biblioteca habilita Cloud Run Admin API, Artifact Registry API, Cloud SQL Admin API, Secret Manager API, IAM API, IAM Service Account Credentials API, Security Token Service API y Cloud Resource Manager API. Este flujo no utiliza Cloud Build.

En IAM y administración > Cuentas de servicio crea storeml-api con Cloud SQL Client. Crea storeml-inference sin permisos de base de datos. Conservan las funciones descritas en el manual: API autenticada y motor Python privado. No generes claves JSON.

Los IDs de las cuentas van en minúsculas: storeml-api, storeml-inference y storeml-deploy. El nombre visible puede usar storeML. Si ya creaste una cuenta con otro ID, crea la cuenta con el ID previsto: el ID no se puede renombrar después de crearla.

Crea storeml-deploy para GitHub Actions. En IAM del proyecto dale Cloud Run Developer (roles/run.developer) y Service Usage Consumer (roles/serviceusage.serviceUsageConsumer). Esta cuenta despliega imágenes pero no administra las políticas públicas/privadas de los servicios. La configuración inicial de esos permisos la realiza tu cuenta administradora en el paso 10.

En cada cuenta de ejecución storeml-api y storeml-inference abre Permisos > Otorgar acceso. Agrega el email de storeml-deploy y Service Account User (roles/iam.serviceAccountUser). No concedas ese rol indiscriminadamente a todas las cuentas del proyecto.

El rol para publicar imágenes se concederá sobre un único repositorio de Artifact Registry en el paso 9. El acceso federado de GitHub a storeml-deploy se configura en el paso 8. Esta cuenta no necesita la contraseña SQL ni Secret Manager Secret Accessor.

Tu cuenta administradora necesita permisos para crear la infraestructura y editar IAM. Cloud SQL Studio puede requerir Cloud SQL Studio User además de las credenciales SQL. Los usuarios Gmail de la app no reciben roles de Google Cloud.

3. Crear la instancia PostgreSQL

En Google Cloud abre SQL > Crear instancia > PostgreSQL. Usa ID de instancia storeml-pg. Selecciona PostgreSQL 17 y edición Enterprise. Para esta instalación inicial selecciona Zona de pruebas y personaliza la máquina: núcleo compartido, db-f1-micro, 0,614 GB de memoria, 10 GB SSD y zona única. Comprueba esos valores en el resumen y revisa el costo estimado antes de crearla. El ajuste Zona de pruebas por sí solo no garantiza la máquina mínima.

Configura la contraseña del administrador postgres y guárdala de forma privada. Elige la región donde desplegarás la API, por ejemplo Santiago / southamerica-west1. Abre las opciones de conexiones: habilita Public IP. No agregues 0.0.0.0/0 ni redes autorizadas generales. La API se conectará mediante el conector oficial, con autorización IAM y cifrado.

La IP pública de esta guía se usa para el conector y no requiere exponer una conexión directa por IP al móvil. No configures certificados de cliente manuales ni un modo que los exija. Para utilizar únicamente IP privada sería necesaria conectividad VPC adicional; esa variante no es la configuración de este manual.

En copias de seguridad configura la política de respaldo y recuperación que necesites. Para una instalación inicial puedes usar una única zona; revisa la disponibilidad requerida antes de producción. Crea la instancia y espera su estado disponible.

En Información general copia el Connection name completo, con formato PROYECTO:REGIÓN:storeml-pg. No es la dirección IP ni el nombre de la base. Lo usarás como INSTANCE_CONNECTION_NAME de la API.

Cloud SQL tiene cargos de instancia, capacidad y almacenamiento aunque la app no reciba solicitudes. Mínimo cero instancias en Cloud Run no apaga PostgreSQL. En Facturación crea un presupuesto y alertas; el aviso no detiene automáticamente el gasto. No se fijan precios en este documento.

4. Crear la base y las cuatro tablas

Dentro de la instancia abre Bases de datos > Crear base de datos. Escribe storeml_database y confirma. En Usuarios > Agregar cuenta crea el usuario de base de datos storeml_app con una contraseña diferente a postgres. Guárdala: se almacenará como secreto en el paso 5.

Abre Cloud SQL Studio en el menú de la instancia. Selecciona autenticación integrada de base de datos (Built-in), base storeml_database, usuario postgres y su contraseña. Elige Nueva consulta. Abre localmente sql/01-esquema.sql, copia todo su contenido al editor y pulsa Ejecutar. Después ejecuta de la misma forma sql/02-catalogo-y-accesos.sql y, por último, sql/03-privilegios-app.sql.

El primer archivo crea las cuatro tablas con claves foráneas y restricciones. El segundo carga las 24 fichas con sus rutas y 12 permisos reservados para tu Gmail. No crea un usuario Google ficticio. El tercero limita storeml_app: puede registrar identidades verificadas, gestionar solicitudes y cuota, pero no otorgar permisos ni editar el catálogo. No uses postgres como usuario de la API.

El tercer archivo retira la membresía cloudsqlsuperuser que Cloud SQL puede conceder al crear usuarios integrados. También quita privilegios de creación de bases/roles y asigna los permisos SQL concretos de la aplicación. Ejecútalo antes de desplegar la API.

En Studio comprueba las tablas con:

```
SELECT table_name FROM information_schema.tables
WHERE table_schema='public' AND table_type='BASE TABLE'
ORDER BY table_name;
```

Deben aparecer modelos, permisos, solicitudes y usuarios. El contador de cuota se guarda en usuarios: no hay una quinta tabla. Si ejecutas de nuevo los archivos iniciales, las tablas existentes y los datos ya presentes se conservan. No restauran automáticamente permisos que hayas deshabilitado.

5. Guardar la contraseña y preparar la conexión

Abre Secret Manager > Crear secreto. Nombre: storeml-db-password. Como valor escribe únicamente la contraseña de storeml_app, sin comillas y sin salto de línea adicional. Guarda el secreto y anota la versión, por ejemplo 1.

En los permisos de ese secreto agrega el email completo de storeml-api@TU_PROYECTO.iam.gserviceaccount.com y el rol Secret Manager Secret Accessor (roles/secretmanager.secretAccessor). Limita el acceso al secreto necesario; el motor de calculadoras no necesita esta contraseña.

La API utiliza estas variables públicas de configuración y un secreto:

```
INSTANCE_CONNECTION_NAME = PROYECTO:REGIÓN:storeml-pg
DB_NAME                  = storeml_database
DB_USER                  = storeml_app
GOOGLE_WEB_CLIENT_ID     = cliente Web OAuth real
INFERENCE_URL            = URL HTTPS del motor privado
DB_PASSWORD              = referencia al secreto, versión 1
```

DB_PASSWORD se configura en Cloud Run como Referencia a un secreto y Exponer como variable de entorno, no pegando la contraseña en el formulario de variables ordinarias. No va en GitHub, en mobile/src/config.ts ni en el PDF. La contraseña de postgres tampoco se utiliza en el contenedor.

El conector oficial usa la identidad de ejecución storeml-api y su rol Cloud SQL Client para acceder a la instancia. El usuario y contraseña PostgreSQL determinan qué operaciones SQL permite esa conexión. Son controles distintos y ambos deben estar configurados.

La API mantiene como máximo cinco conexiones SQL por instancia de Cloud Run. Con máximo tres instancias habrá hasta quince conexiones de aplicación, además de las administrativas. El servidor valida la conexión al iniciar; un nombre de conexión, usuario o contraseña incorrecto puede impedir que la revisión arranque.

6. Configurar Google Sign-In

En Google Auth Platform abre Branding y completa AutoMind, email de soporte y datos requeridos. En Audience añade artemioaday@gmail.com a Test users durante las pruebas. En Data Access utiliza identidad básica: openid, email y profile. Esta app no requiere Firebase Authentication.

En Clients > Create client crea Web application. Guarda su Client ID terminado en .apps.googleusercontent.com. Ese mismo ID va en GOOGLE_WEB_CLIENT_ID de la API y googleWebClientId de mobile/src/config.ts. No se usa el secreto OAuth Web en el móvil.

Crea un cliente de tipo Android con package name com.storeml.app y el SHA-1 del certificado que firmará la app. Para el APK debug de este ZIP la huella SHA-1 verificada es 5E:8F:16:06:2E:A3:CD:2C:4A:0D:54:78:76:BA:A6:F3:8C:AB:F6:25. El paso 13 explica cómo obtenerla con Android Studio. Registra los certificados debug y release que utilices; para Play App Signing añade también el certificado de firma mostrado en Play Console.

Para iOS crea un cliente con bundle ID com.automind. Copia su Client ID a googleIOSClientId y el iOS URL scheme a mobile/ios/AutoMind/Info.plist, reemplazando REEMPLAZAR_REVERSED_IOS_CLIENT_ID. El cliente Android, iOS y Web deben corresponder al proyecto adecuado.

La API verifica firma, audiencia, emisor, expiración y Gmail verificado. El registro usa el subject estable que acredita Google. Ninguna ruta acepta otro Gmail enviado en JSON para decidir permisos. El usuario autorizado puede consultar la Store y solicitar modelos; cada suma exige además su permiso específico.

7. Subir el repositorio y obtener sus IDs

Crea o utiliza un repositorio público en GitHub. Sube el contenido extraído de AutoMind, con api, inference, sql, mobile, catalog, infra y docs directamente en la raíz. Incluye .gitignore y la carpeta oculta .github. El workflow debe quedar en .github/workflows/deploy.yml.

Si subes por lotes usando Add file > Upload files, carga .github al final para que las pruebas ya encuentren las fuentes. Mantén main como rama principal. No subas el ZIP como archivo ni contraseñas, claves o resultados de compilación.

En GitHub > Actions abre AutoMind - pruebas y despliegue > Run workflow, selecciona main y activa solo_identidad. Ese modo imprime datos del repositorio y omite pruebas/despliegue. En el job identidad > Datos del repositorio copia REPOSITORY_ID y OWNER_ID. Son números de GitHub, distintos a los IDs de ocho dígitos de tus modelos.

En un push inicial sin configurar las variables, las pruebas pueden ejecutarse y el despliegue se detendrá al validar la configuración. No se autentica en Google ni crea recursos hasta completar los pasos siguientes.

Cada modelo conserva un archivo Python, por ejemplo inference/modelos/inmobiliario/cart.py, con MODEL_ID=10000002. La columna archivo contiene esa ruta relativa. El contenedor ejecuta una copia desplegada y el registro fijo controla los módulos permitidos. No descarga código al recibir una petición.

Si ya usabas triggers de Cloud Build para estos servicios, desactívalos desde Cloud Build > Triggers antes de activar este flujo, para que un mismo commit no tenga dos sistemas de despliegue.

8. Configurar OIDC / Workload Identity Federation

En Google Cloud busca Workload Identity Federation dentro de IAM y administración. Crea un pool llamado GitHub, ID github, habilitado. Crea en él un proveedor OIDC con ID storeml. Issuer URL: https://token.actions.githubusercontent.com. Conserva las audiencias predeterminadas; la acción utiliza el recurso completo del proveedor como audiencia.

En el mapeo de atributos agrega estas parejas. La columna izquierda es el atributo Google y la derecha su valor:

```
google.subject                 assertion.sub
attribute.repository_id        assertion.repository_id
attribute.repository_owner_id  assertion.repository_owner_id
attribute.ref                  assertion.ref
attribute.event_name           assertion.event_name
```

En Attribute condition pega la siguiente expresión, sustituyendo OWNER_ID y REPOSITORY_ID por los números del paso 7, dentro de las comillas:

```
assertion.repository_owner_id == 'OWNER_ID' &&
assertion.repository_id == 'REPOSITORY_ID' &&
assertion.ref == 'refs/heads/main' &&
(assertion.event_name == 'push' ||
 assertion.event_name == 'workflow_dispatch')
```

Guarda. Copia el nombre completo del proveedor: projects/NUMERO_PROYECTO/locations/global/workloadIdentityPools/github/providers/storeml. Usa el número del proyecto Google, no su ID de texto.

En el pool usa Otorgar acceso mediante una cuenta de servicio, selecciona storeml-deploy y restringe las identidades a attribute.repository_id igual al REPOSITORY_ID. El permiso resultante es Workload Identity User (roles/iam.workloadIdentityUser) sobre storeml-deploy.

También puedes agregarlo en los permisos de esa cuenta con el principal: principalSet://iam.googleapis.com/projects/NUMERO_PROYECTO/locations/global/workloadIdentityPools/github/attribute.repository_id/REPOSITORY_ID. No concedas acceso al pool entero. El proveedor añade las restricciones de propietario, rama y evento indicadas arriba. La configuración puede tardar unos minutos en propagarse.

Si aparece Configura tu aplicación y pide Ruta de acceso OIDC ID token, cierra únicamente ese formulario de descarga. El permiso guardado se conserva. GitHub Actions obtiene el token automáticamente mediante la acción auth; no necesita una ruta local de token ni descargar ADC manualmente. Verifica en Cuentas de servicio conectadas que aparezca storeml-deploy con el filtro repository_id del repositorio.

9. Artifact Registry y variables de GitHub

En Google Cloud > Artifact Registry > Crear repositorio, usa nombre storeml, formato Docker, modo Standard y región southamerica-west1 (o la región que elijas para los servicios). En los permisos de ese repositorio concede Artifact Registry Writer a storeml-deploy. El repositorio debe existir antes del primer despliegue.

En GitHub > repositorio > Settings > Secrets and variables > Actions > Variables, crea estas variables de repositorio. Son identificadores de configuración, no claves privadas:

```
GCP_PROJECT_ID               ID de texto del proyecto Google
GCP_WIF_PROVIDER             Nombre completo del proveedor OIDC
GCP_DEPLOY_SERVICE_ACCOUNT   Email de storeml-deploy
INSTANCE_CONNECTION_NAME    PROYECTO:REGIÓN:storeml-pg
GOOGLE_WEB_CLIENT_ID         Cliente OAuth Web real
GCP_REGION                  southamerica-west1
ARTIFACT_REPOSITORY          storeml
DB_PASSWORD_SECRET          storeml-db-password
DB_PASSWORD_SECRET_VERSION  1
```

Las últimas cuatro tienen los valores predeterminados mostrados si las omites. Aun así, conviene definirlas explícitamente. Si tu secreto tiene otra versión numérica, usa esa versión. No uses latest ni pegues la contraseña SQL como variable o secreto de GitHub.

Los servicios y usuarios tienen nombres fijos: automind-api, automind-inference, DB_NAME=storeml_database, DB_USER=storeml_app. Las cuentas de ejecución son storeml-api@PROYECTO.iam.gserviceaccount.com y storeml-inference@PROYECTO.iam.gserviceaccount.com. El validador exige que la instancia SQL y la cuenta de despliegue pertenezcan a GCP_PROJECT_ID.

La contraseña permanece en Secret Manager según el paso 5. La API necesita acceso a ese secreto; GitHub solo pasa la referencia al configurar Cloud Run. Actions solicita un token Google temporal por OIDC después de compilar las imágenes; no requiere descargar una clave JSON.

10. Primer despliegue y permisos iniciales

En GitHub Actions ejecuta AutoMind - pruebas y despliegue sobre main con solo_identidad desactivado. También se ejecuta automáticamente con cualquier push a main. El job pruebas levanta PostgreSQL 17 temporal, ejecuta las 22 pruebas Python de API, las calculadoras y la validación de variables. Solo si pasan comienza el despliegue.

El job desplegar comprueba que el commit sigue siendo el actual de main, compila las dos imágenes, obtiene un token OIDC temporal, publica en Artifact Registry y despliega motor y API por digest. Los despliegues se serializan; los commits antiguos pendientes se omiten. Un despliegue en curso no se cancela a mitad por un nuevo push.

El workflow configura INSTANCE_CONNECTION_NAME, DB_NAME, DB_USER, APP_ENV, GOOGLE_WEB_CLIENT_ID e INFERENCE_URL, y referencia DB_PASSWORD en Secret Manager. No ejecuta los scripts SQL sobre producción: las tablas y datos iniciales deben haberse creado con el paso 4.

Al finalizar, abre Summary y copia las URLs de API y motor. En Cloud Run, revisa automind-inference: debe requerir autenticación. En sus permisos concede Cloud Run Invoker a storeml-api@TU_PROYECTO.iam.gserviceaccount.com. No añadas allUsers al motor.

En automind-api configura Permitir acceso público desde Seguridad/Autenticación. La API seguirá comprobando Google en cada ruta. Los servicios nuevos se crean privados por defecto; esta configuración manual se hace una sola vez. El workflow no cambia esas políticas IAM porque la cuenta de despliegue no tiene permisos para hacerlo.

Abre la URL de API seguida de /v1/catalog en el navegador: debe responder 401 desde la API. Si Cloud Run muestra su propia denegación IAM, revisa que la API permita acceso público. Copia esa URL HTTPS a apiUrl en mobile/src/config.ts. En futuros pushes, las revisiones se actualizarán conservando los permisos de los servicios.

11. Editar catálogo, accesos y solicitudes

Administra datos desde Cloud SQL Studio, entrando a storeml_database como postgres u otro administrador autorizado. No necesitas dar credenciales SQL a los usuarios móviles. Para revisar el catálogo y cambiar un texto:

```
SELECT id,nombre,area,archivo,activo FROM public.modelos ORDER BY id;
UPDATE public.modelos SET descripcion='Suma los tres inputs.'
WHERE id=10000002;
```

Para ocultar y detener un modelo usa UPDATE public.modelos SET activo=FALSE WHERE id=10000002; no borres filas con relaciones. El backend comprueba activo al usarlo. Los IDs iniciales y el detalle de sus archivos están en sql/02-catalogo-y-accesos.sql y catalog/models.json.

Para revisar tus permisos:

```
SELECT email,modelo_id,usuario_id,habilitado FROM public.permisos
WHERE email='artemioaday@gmail.com' ORDER BY modelo_id;
```

Para aprobar una solicitud utiliza una transacción como esta, cambiando el Gmail y el ID por los de la solicitud real:

```
BEGIN;
INSERT INTO public.permisos (email,modelo_id,usuario_id,habilitado)
SELECT email,10000004,google_sub,TRUE FROM public.usuarios
WHERE email='cliente@gmail.com'
ON CONFLICT (email,modelo_id) DO UPDATE SET habilitado=TRUE;
UPDATE public.solicitudes SET estado='approved',
  actualizado_en=CURRENT_TIMESTAMP
WHERE modelo_id=10000004 AND usuario_id=(
  SELECT google_sub FROM public.usuarios WHERE email='cliente@gmail.com');
COMMIT;
```

No cambies solo el estado de una solicitud: la autorización depende de permisos. Para revocar un ID usa UPDATE public.permisos SET habilitado=FALSE WHERE email='cliente@gmail.com' AND modelo_id=10000004;. Para suspender todos los modelos, omite la condición modelo_id. No cambies usuario_id para apuntar a otra cuenta: la clave foránea exige coincidencia de subject y Gmail.

12. Registro y pruebas de aceptación

El archivo inicial reserva a artemioaday@gmail.com los IDs 10000001, 10000002, 10000003, 10000007, 10000008, 10000009, 10000013, 10000014, 10000015, 10000019, 10000020 y 10000021. Son 12 modelos, tres por área. usuario_id permanece NULL antes del registro y se completa con el subject del primer login Google verificado.

Configura la app, compila y entra con tu Gmail. Mis modelos debe mostrar esos 12 y la Store los 24 activos. Abre 10000002 e introduce 10, -3 y 0,5. El resultado debe ser 7,5 unidades. El cálculo se ejecuta desde el archivo de ese ID.

Comprueba en Studio que usuarios contiene tu google_sub y email reales y que tus 12 permisos están vinculados. Las solicitudes solo aparecen cuando se envían desde la app. Ninguna solicitud pendiente concede acceso por sí misma.

Prueba con otro Gmail autorizado como Test user en OAuth: debe poder iniciar sesión y consultar Store, pero no calcular sin permisos. Solicita un ID desde esa cuenta, revisa solicitudes y apruébalo con el paso 11. Revócalo y verifica que la siguiente petición se deniega.

El servidor vuelve a leer PostgreSQL antes de llamar al motor y antes de entregar el resultado. Los resultados ya entregados no se pueden retirar. La cuota transaccional es de 30 cálculos por minuto por subject, en las columnas cuota_minuto y cuota_usos de usuarios.

En Cloud Run > Logs revisa fallos. 401: sesión/cliente Web/Gmail; 403: permiso de ID o IAM del motor; 400: inputs o formato ID; 429: cuota; 503: motor. Un 500 requiere revisar SQL, esquema, privilegios y conexión. No registres tokens ni contraseñas.

La API incluye 22 pruebas Python. Las ocho de integración requieren TEST_DATABASE_URL hacia una base PostgreSQL de prueba vacía; las demás se ejecutan con repositorio y verificador inyectados. En esta entrega se probaron las consultas mediante pg8000 y un motor PGlite local de QA. La red de Cloud SQL, OAuth e IAM deben comprobarse en tu proyecto. Consulta docs/VERIFICACION.md.

13. Compilar Android en tu equipo

Instala Node.js 22 LTS actualizado, JDK 17 y Android Studio. En Android Studio > SDK Manager instala Android SDK Platform 36, Build Tools 36.0.0, NDK 27.1.12297006 y CMake 3.22.1. Extrae el proyecto en una ruta corta, por ejemplo C:\dev\AutoMind. Esta sección usa herramientas locales; la configuración de Google Cloud de los pasos anteriores se realiza con interfaz.

Desde la terminal local de la carpeta mobile ejecuta npm ci para instalar las librerías. Abre mobile/android en Android Studio. Selecciona el JDK 17 para Gradle y deja terminar la sincronización. Si no encuentra el SDK, configura su ubicación desde las opciones de Android Studio; Android puede crear el local.properties correspondiente.

Para el SHA-1 de debug abre el panel Gradle > app > Tasks > android > signingReport y ejecútalo. Si no se muestra esa tarea, desde la terminal local de mobile/android ejecuta .\gradlew.bat signingReport. Copia el SHA-1 de la variante debug al cliente OAuth Android del paso 6.

Para desarrollo, desde mobile ejecuta npm start y, en otra terminal, npm run android con un dispositivo conectado o emulador con Google Play Services. El debug utiliza Metro. Esta entrega ya configura mobile/src/config.ts con la URL HTTPS de automind-api y el ID OAuth Web de MLStore. El applicationId y namespace de Android son com.storeml.app; deben coincidir con el cliente Android creado en Google. El identificador del componente React Native continúa siendo AutoMind.

Para un APK/AAB release, en Android Studio usa Build > Generate Signed App Bundle / APK > Create new para crear tu clave privada. Anota la ruta, alias y contraseñas y guarda una copia segura. Configura las cuatro propiedades AUTOMIND_STORE_FILE, AUTOMIND_STORE_PASSWORD, AUTOMIND_KEY_ALIAS y AUTOMIND_KEY_PASSWORD en tu archivo personal ~/.gradle/gradle.properties; no las agregues al repositorio.

Desde mobile/android puedes ejecutar .\gradlew.bat assembleRelease para APK y .\gradlew.bat bundleRelease para AAB. Quedan en app/build/outputs/apk/release y app/build/outputs/bundle/release. El release incluye el bundle y no utiliza Metro. Registra su SHA-1. Si usas Play App Signing, añade también el SHA-1 de la clave de firma de Play y completa los formularios de Play Console.

La compilación nativa anterior se detuvo por espacio de disco; esta entrega no afirma haber generado un APK. El detalle de las comprobaciones está en docs/VERIFICACION.md. Usa un equipo con espacio suficiente para SDK, cachés y compilación.

14. Migración, respaldo e iOS

Si antes subiste la API TypeScript al repositorio, sustituye sus fuentes por la carpeta api actual. Elimina del repositorio los antiguos api/src, api/test, package.json, package-lock.json y tsconfig.json de esa API; conserva los archivos TypeScript de mobile. El nuevo Dockerfile utiliza Python y los endpoints no cambian. No se modifica el esquema PostgreSQL por este cambio de lenguaje.

Si aún no configuraste la nube, sigue este manual directamente. Ya no se utilizan las cuatro bases Firestore ni reglas Firebase. El ZIP nuevo sustituye el backend de documentos por PostgreSQL y no requiere instalar Firebase.

Si tienes datos reales en Firestore, consérvalos mientras preparas PostgreSQL. El SQL inicial solo contiene catálogo y tus 12 permisos; no importa automáticamente otros usuarios, accesos o solicitudes. Revisa esos datos y cópialos deliberadamente antes de cambiar la revisión que atiende la app.

Los permisos adicionales pueden reservarse por Gmail con usuario_id NULL y vincularse cuando esa cuenta entre de nuevo. Para conservar solicitudes antiguas debes copiar primero el subject y email del registro Google real a usuarios, después su modelo y estado a solicitudes. No inventes un subject. Compara los registros y prueba ambas cuentas antes de retirar recursos anteriores.

En Cloud SQL > Copias de seguridad puedes crear un respaldo manual antes de cambios. Para volver atrás en código utiliza las revisiones de Cloud Run; una revisión anterior no deshace cambios SQL. En esta entrega no se borran tablas ni se eliminan tus recursos de nube.

Para rotar la contraseña, cambia storeml_app en Cloud SQL, añade una versión del secreto, actualiza DB_PASSWORD_SECRET_VERSION en las variables de GitHub y ejecuta el workflow. No cambies solo la revisión desde la consola: el siguiente push vuelve a aplicar las variables de GitHub. Coordina la rotación para evitar cortes.

iOS requiere un Mac con Xcode. Desde mobile ejecuta npm ci, bundle install y, dentro de ios, bundle exec pod install. Abre AutoMind.xcworkspace, configura Team, firma, bundle ID com.automind, cliente iOS y URL scheme del paso 6. Ejecuta o archiva desde Xcode. No se ha compilado iOS en este entorno Windows.

15. Operación automática y errores

En cada push a main, incluso cambios de documentación o mobile, se ejecutan las pruebas y se despliegan los dos servicios backend si pasan. Las pull requests solo prueban. Este workflow no compila ni publica APK, AAB o iOS; esos pasos del móvil siguen siendo los de la sección 13.

La base temporal de CI usa una contraseña de prueba sin relación con Cloud SQL. Las pruebas de integración necesitan una base vacía y nunca se conectan a producción. Los tres SQL iniciales se aplican administrativamente desde Studio; un cambio posterior de esquema requiere su revisión y aplicación manual. Subir un archivo SQL no lo ejecuta en producción.

Si falla Validar variables, revisa los nombres en Settings > Secrets and variables > Actions > Variables. Si falla OIDC, revisa proveedor, IDs numéricos del propietario/repositorio, rama main y Workload Identity User. Si el error es al publicar imágenes, comprueba Artifact Registry Writer y la región del repositorio.

Si Cloud Run no permite desplegar, revisa Cloud Run Developer y Service Account User sobre ambas cuentas de ejecución. Si la API no arranca, revisa el secreto, su versión, Cloud SQL Client y los SQL del paso 4. La URL del motor se toma automáticamente de su servicio desplegado.

Protege quién puede actualizar main desde Settings > Rules/Branches. El proveedor solo confía en ese repositorio y esa rama, pero cualquier código autorizado que llegue a main será probado y desplegado. Las acciones externas están fijadas por SHA de commit; actualiza esas referencias deliberadamente y vuelve a validar el workflow.

Las dos revisiones se despliegan secuencialmente, no como una transacción única. Si falla la API después de actualizar el motor, revisa el build y vuelve a ejecutar el workflow; los cambios incompatibles entre ambos servicios requieren planificar versiones. Cloud Run permite volver a una revisión anterior desde su consola. Los cambios SQL se gestionan por separado.

16. Archivos y referencias

sql/01-esquema.sql: cuatro tablas, claves foráneas, IDs de ocho dígitos, estados y cuota.

sql/02-catalogo-y-accesos.sql: 24 modelos con ruta archivo y tus 12 permisos. Conserva cambios previos al repetirlo.

sql/03-privilegios-app.sql: rol limitado storeml_app. El backend no puede conceder acceso ni modificar los modelos.

inference/modelos/AREA/MODELO.py: las 24 sumas independientes, con MODEL_ID numérico. inference/registry.py: relación fija entre ID y módulo permitido.

api/automind_api/repository.py: consultas SQL parametrizadas. api/automind_api/database.py: conector Python oficial y pool SQLAlchemy. api/automind_api/main.py inicia la API FastAPI. mobile/src/config.ts: URL de API y clientes OAuth públicos, sin secretos.

El despliegue está en .github/workflows/deploy.yml. El código de validación de variables es infra/github_config.py.

Referencias oficiales consultadas:

Cloud SQL PostgreSQL: https://docs.cloud.google.com/sql/docs/postgres/create-instance

Cloud SQL Studio: https://docs.cloud.google.com/sql/docs/postgres/manage-data-using-studio

Usuarios y roles SQL: https://docs.cloud.google.com/sql/docs/postgres/create-manage-users

Conector oficial Python: https://github.com/GoogleCloudPlatform/cloud-sql-python-connector

Secret Manager: https://docs.cloud.google.com/secret-manager/docs/create-secret-quickstart

Secretos en Cloud Run: https://docs.cloud.google.com/run/docs/configuring/services/secrets

Repositorio y despliegue: https://docs.cloud.google.com/run/docs/continuous-deployment

Google OAuth: https://react-native-google-signin.github.io/docs/setting-up/get-config-file

Restricciones SQL: https://www.postgresql.org/docs/current/ddl-constraints.html

Los menús pueden aparecer traducidos. Los pasos describen la configuración desde interfaz y el editor de Studio. No son capturas ni prueba de un despliegue ejecutado en tu cuenta. La entrega incluye fuentes y guía; los recursos, secretos y login reales se configuran con este procedimiento.

GitHub y OIDC: https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-google-cloud-platform

Workload Identity Federation: https://docs.cloud.google.com/iam/docs/workload-identity-federation-with-deployment-pipelines